Beatwave

Fabó Dudás Róbert, Muzsik Csilla

Tartalom

Projektleírás ... 3
Célközönség ... 3
Funkciók ... 4
Spotify fiók csatlakoztatása a profilban ... 4
Top előadók és számok megjelenítése ... 4
Aktuálisan játszott zene megjelenítése ... 4
Kereső funkció ... 4
Közösségi beszélgetések (Discussion) ... 4
Fejlesztői dokumentáció ... 5
Miért volt rá szükség? ... 5
Fejleszőkörnyezet ... 5
Hardver ... 5
Használt eszközök ... 5
Választott programozási nyelv ... 6
Adatbázis felépítése ... 7
Szerkezete ... 7
Táblák részletes ismertetése ... 7
Algoritmusok ... 12
formatRelative (relatív idő formázása) ... 12
getNormalizedHashtags (hashtag normalizálása) ... 12
parseCookieHeader (cookie fejléc feldolgozása) ... 13
getBodyParserFriendlyError (body-parser hibák leképezése és formázása) ... 13
Tesztelés ... 14
Login teszt (POST /login) ... 14
Regisztrációs teszt (POST /register) ... 14
Felhasználói profil lekérdezés (GET /user-profile) ... 15
Új hozzászólás létrehozása (POST /posts) ... 15
Jelszó visszaállítási kérés ismeretlen emailre (POST /auth/password-reset/request) ... 16
Hirdetések lekérdezése (GET /announcements) ... 16
Hibakezelés ... 17

getUniqueConstraintMessage (egyedi megszorítási hibaüzenet generálása) 17
sanitizeClientMessage (kliensnek szánt hibaüzenet tisztítása és szűrése) 17
errorHandler (központi Express hibakezelő middleware) 18
Fejlesztési lehetőségek 19
Képek föltöltése 19
Apple Music 19
Alkalmazás készítése 19
Felhasználói dokumentáció 20
Projekt áttekintése a felhasználók számára 20
Szükséges hardver és szoftver 20
Elérés 20
Az oldal használata 21
Funkciók elérése 22
Beviteli mezők 22
1. Fiók- és bejelentkezési adatok (Authentication Fields) 22
2. Közösségi funkciók (Create a post) 23
3. Profil személyre szabása (Edit profile) 24
Hibajelzések 25
Megjelenés és vizuális jellemzők 25
A hibajelzések típusai és kiváltó okaik 25
Admin lehetőségei 28
Kereső 28
Összefoglalás 30
Értékelés 31
Irodalomjegyzék 32

Projektleírás

A Beatwave egy modern, zenealapú közösségi platform, amely összekapcsolja a felhasználók zenei ízlését és közösségi élményeit. Az oldal fő funkciója, hogy a Spotify-fiókokból származó adatok alapján megjeleníti a felhasználó leggyakrabban hallgatott zenéit, előadóit és zenei műfajait. Így mindenki átláthatja saját zenei szokásait egy esztétikus, személyre szabott felületen. A rendszer dinamikusan frissül, így mindig a legaktuálisabb toplistákat mutatja, legyen szó heti, havi vagy összesített statisztikákról.

A Beatwave nemcsak adatvizualizációs eszköz, hanem közösségi tér is: a felhasználók különböző témájú beszélgetésekben vehetnek részt kedvenc előadóikról, albumokról vagy akár zenei trendekről. A chatfunkció hasonlóan működik, mint az Instagram Threads, így gyors, interaktív és személyes kommunikációra ad lehetőséget.

A projekt célja, hogy egyesítse a zenei streaming platformok adatainak elemzését a közösségi élménnyel, ezáltal egy olyan digitális helyet hozzon létre, ahol a zene szerelmesei nemcsak hallgatják, hanem megosztják és megvitatják is a zenét, ami inspirálja őket.

Célközönség

Első sorban olyan felhasználóknak, akik sűrűn hallgatnak zenét Spotify-on, és kíváncsiak a statisztikájukra. Emellett azoknak is szól, akik szeretnének a kedvenc előadójukról vagy zenéjükről más rajongokkal beszélgetni, véleményt megosztani.

Tapasztalataink szerint, nem sok olyan oldal vagy applikáció van, ahol ezeket az adatokat meg lehet tekinteni. A mi oldalunk célja, hogy ezt díjmentesen és egyszerűen megtehessék a felhasználók, ezzel együtt egy olyan platformot létrehozni, ahol a zenehallgatók szabadon tudnak kommunikálni egymással. Mindemellett zenékre is lehet keresni.

Funkciók

Spotify fiók csatlakoztatása a profilban

A felhasználók a profiloldalukon keresztül tudják összekapcsolni Spotify fiókjukat a Beatwave platformmal. Ez a csatlakozás szükséges ahhoz, hogy a rendszer hozzáférjen a felhasználó zenei adataihoz, például a leggyakrabban hallgatott előadókhoz és számokhoz. A csatlakoztatás OAuth 2.0 protokollon keresztül történik, amely biztosítja a biztonságos és engedélyezett adatlekérést a Spotify API-tól.

Top előadók és számok megjelenítése

Miután a Spotify fiók csatlakoztatva van, a főoldalon megjelennek a felhasználó legnépszerűbb előadói és zeneszámai. Ezek az adatok a Spotify API által szolgáltatott statisztikák alapján frissülnek dinamikusan, így a felhasználók mindig a legaktuálisabb toplistákat láthatják. Az adatok megjelenítése esztétikus, személyre szabott felületen történik, amely vizuálisan is vonzó és könnyen áttekinthető.

Aktuálisan játszott zene megjelenítése

Ha a felhasználó Spotify-ján éppen zene szól, a Beatwave automatikusan lekéri és megjeleníti az aktuálisan játszott szám adatait (előadó neve, szám címe, borítókép). Ez a funkció valós idejű adatfrissítést igényel, amely a Spotify API „currently playing” végpontját használja. Ezáltal a felhasználók mindig láthatják, milyen zenét hallgatnak, akár a közösségi térben is megoszthatják ezt az információt.

Kereső funkció

A felső menüsorban található keresőmező lehetővé teszi, hogy a felhasználók zenékre, előadókra vagy albumokra keressenek rá. A keresés valós idejű, és a Spotify API kereső végpontját használja, így a találatok gyorsan és relevánsan jelennek meg. Ez a funkció segíti a felhasználókat abban, hogy könnyen megtalálják kedvenc zenéiket vagy új előadókat fedezzenek fel.

Közösségi beszélgetések (Discussion)

A Beatwave nem csupán adatvizualizációs eszköz, hanem közösségi platform is. A „Discussion” menüpont alatt a felhasználók különböző témájú beszélgetésekben vehetnek részt, ahol megoszthatják véleményüket kedvenc előadóikról, albumokról vagy zenei trendekről. A chat funkció hasonló az Instagram Threads-hez, gyors, interaktív és személyes kommunikációt tesz lehetővé. Ez a közösségi tér elősegíti a zene iránti szenvedély megosztását és a felhasználók közötti kapcsolatok építését.

Fejlesztői dokumentáció

Miért volt rá szükség?

A Beatwave fejlesztésére azért volt szükség, mert a piacon elérhető zenei statisztika- és közösségi platformok, mint például a stats.fm, nem nyújtanak teljes körű és hibamentes élményt a felhasználók számára. Weboldalunk lehetővé teszi, hogy több zenei platformról (például a Spotify-ról) is lehessen zenékre keresni, így a felhasználók szélesebb körű adatokat érhetnek el egy helyen. Ezen túlmenően a Beatwave pontosan és hiba nélkül jeleníti meg a felhasználók által hallgatott zenéket, ellentétben a stats.fm-mel, ahol előfordulhatnak pontatlanságok vagy adatmegjelenítési hibák. Ez a megbízhatóság és a többplatformos keresési lehetőség egyedivé és versenyképessé teszi a programot. Továbbá a Beatwave nem csupán adatvizualizációs eszköz, hanem egy interaktív közösségi tér is, ahol a felhasználók megoszthatják zenei élményeiket, véleményüket, és aktívan részt vehetnek beszélgetésekben, ami a hasonló szolgáltatásokban ritkán található meg ilyen formában.

Fejleszőkörnyezet

Hardver

- Processzor: Intel Core i3-7100 (3.90 GHz)
- Grafikus kártya: Intel(R) HD Graphics 630 (Integrált)
- Memória (RAM): 16 GB DDR4
- Tárhely: 512 GB SSD/HDD
- Operációs Rendszer: Windows 10

Használt eszközök

A projekt fejlesztése során az alábbi eszközöket és szoftvereket használtuk, amelyek hozzájárultak a hatékony munkavégzéshez és a magas minőségű végeredmény eléréséhez:

Szoftverek

- Visual Studio Code
- GitHub Desktop
- XAMPP

Frontend

- Vite React
- Tailwind CSS
- CSS

Backend

- Express.js
- Node.js

Adatbázis

- Prisma
- MySQL

Választott programozási nyelv

A TypeScript legnagyobb előnye számunkra a típusbiztonság, amely már a gépelés közben, jóval a kód futtatása előtt segít elkapni a végzetes hibákat. Ezáltal rengeteg idegesítő hibakereséstől és böngészős összeomlástól kímél meg minket. Ehhez szorosan kapcsolódik a kiváló fejlesztői élmény is: a kódszerkesztőnk a típusok ismeretében automatikusan felkínálja a megfelelő kiegészítéseket (IntelliSense), ami jelentősen felgyorsítja és kényelmessé teszi a munkánkat. Ahogy a projekt egyre nagyobbra nő, ez a szigorúság számunkra egy életmentő védőhálóvá válik, mert egy-egy nagyobb átalakításnál (refaktorálásnál) a rendszer azonnal megmutatja, mely pontokon tört el a kód a módosításaink miatt. Ráadásul az előre definiált típusok öndokumentálóvá teszik a kódot, így hetek vagy hónapok múltán is azonnal, külső magyarázat nélkül értjük, hogy melyik függvény pontosan milyen adatot vár és mit ad vissza

.

Adatbázis felépítése

Szerkezete

Tábla neve	Funkció
user	Felhasználók adatai, hitelesítés, jogosultságok
post	Felhasználók által létrehozott posztok
comment	Posztokhoz vagy más kommentekhez fűzött hozzászólások
commentlike	Kommentekhez tartozó kedvelések
notification	Felhasználói értesítések
moderationlog	Moderációs események naplózása, státuszokkal
connectedapp	Külső platformokhoz kapcsolódó alkalmazások és hozzáférési tokenek
postlike	Posztokhoz tartozó kedvelések
revokedtoken	Visszavont hozzáférési tokenek
_prisma_migrations	Az adatbázis migrációk nyomon követése (Prisma ORM használatával)

Táblák részletes ismertetése

user

- Leírás: A rendszer felhasználóinak adatait tárolja.
- Mezők:
  - id (varchar(191), PK): Egyedi azonosító
  - email (varchar(191)): Felhasználó email címe
  - username (varchar(191)): Felhasználónév
  - passwordHash (varchar(191)): Jelszó hash értéke
  - role (enum: 'USER', 'MODERATOR', 'ADMIN'): Felhasználói szerepkör
  - createdAt (datetime(3)): Regisztráció időpontja
  - updatedAt (datetime(3)): Utolsó módosítás időpontja
  - description (text): Felhasználói leírás

    - isPrivate (tinyint(1)): Privát profil státusz
    - isBlocked (tinyint(1)): Tiltott státusz
    - spotifyTimeRange (enum: 'SHORT', 'MEDIUM', 'LONG'): Spotify időtartomány beállítása

post

- Leírás: Felhasználók által létrehozott posztok.
- Mezők:
    - id (varchar(191), PK): Egyedi posztazonosító
    - text (longtext): Poszt szövege
    - likeAmount (int): Kedvelések száma
    - hashtags (varchar(191)): Poszthoz kapcsolódó hashtagek
    - postedAt (datetime(3)): Poszt létrehozásának időpontja
    - userId (varchar(191), FK): Poszt készítőjének azonosítója
    - title (varchar(191)): Poszt címe
    - topic (varchar(191)): Poszt témája

comment

- Leírás: Posztokhoz vagy más kommentekhez fűzött hozzászólások.
- Mezők:
    - id (varchar(191), PK): Egyedi kommentazonosító
    - text (text): Komment szövege
    - likeAmount (int): Kedvelések száma
    - commentedAt (datetime(3)): Komment létrehozásának időpontja
    - userId (varchar(191), FK): Komment írójának azonosítója
    - postId (varchar(191), FK): A kommenthez tartozó poszt azonosítója
    - previousCommentId (varchar(191), FK, NULL): Ha válasz egy másik kommentre, annak azonosítója

commentlike

- Leírás: Kommentekhez tartozó kedvelések.
- Mezők:
  - id (varchar(191), PK): Egyedi azonosító
  - userId (varchar(191), FK): A kedvelő felhasználó azonosítója
  - commentId (varchar(191), FK): A kedvelt komment azonosítója

notification

- Leírás: Felhasználói értesítések kezelése.
- Mezők:
  - id (varchar(191), PK): Egyedi értesítésazonosító
  - type (varchar(191)): Értesítés típusa (pl. like, comment, follow)
  - read (tinyint(1)): Olvasottsági státusz (0 = olvasatlan, 1 = olvasott)
  - createdAt (datetime(3)): Értesítés létrehozásának időpontja
  - userId (varchar(191), FK): Az értesített felhasználó azonosítója
  - triggeredById (varchar(191), FK, NULL): Az értesítést kiváltó felhasználó azonosítója
  - message (text): Értesítés szövege
  - link (varchar(191)): Kapcsolódó link (pl. posztra mutató URL)

moderationlog

- Leírás: Moderációs események naplózása.
- Mezők:
  - id (varchar(191), PK): Egyedi naplóbejegyzés azonosító
  - status (enum: 'DELETED', 'NOTIFIED', 'REPORTED', 'DISMISSED', 'BLOCKED', 'ROLE_CHANGED'): Moderáció státusza
  - createdAt (datetime(3)): Naplózás időpontja
  - postId (varchar(191), FK, NULL): Érintett poszt azonosítója
  - commentId (varchar(191), FK, NULL): Érintett komment azonosítója
  - userId (varchar(191), FK): Moderátor vagy érintett felhasználó azonosítója
  - action (varchar(191)): Moderációs művelet

    - details (varchar(191)): Részletek
    - moderatorId (varchar(191)): Moderátor azonosítója

connectedapp

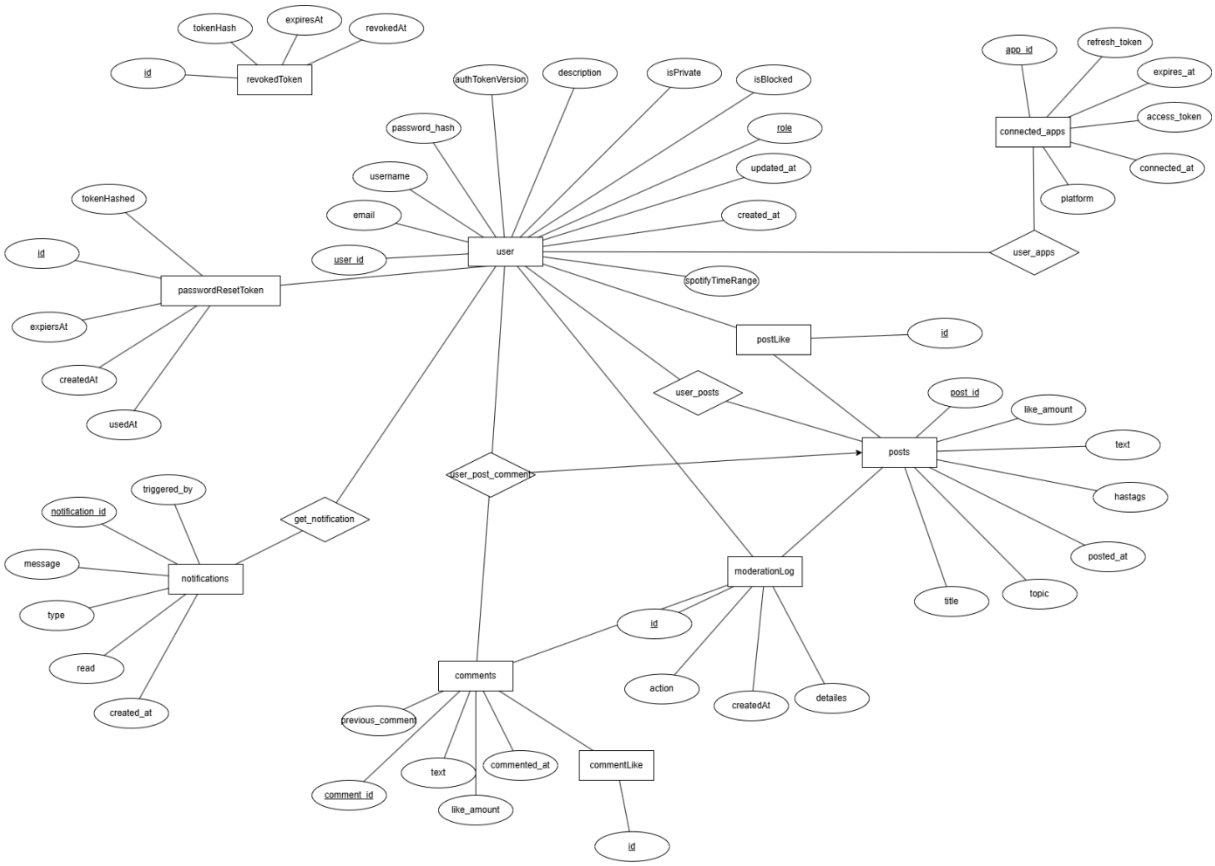
- Leírás: Külső platformokhoz kapcsolódó alkalmazások és hitelesítési adatok.
- Mezők:
    - id (varchar(191), PK): Egyedi azonosító
    - accessToken (text): Hozzáférési token
    - connectedAt (datetime(3)): Kapcsolódás időpontja
    - userId (varchar(191), FK): Felhasználó azonosítója
    - expiresAt (datetime(3)): Token lejárati ideje
    - platform (varchar(191)): Külső platform neve (pl. Facebook, Google)
    - refreshToken (text): Frissítő token

postlike

- Leírás: Posztokhoz tartozó kedvelések.
- Mezők:
    - id (varchar(191), PK): Egyedi azonosító
    - userId (varchar(191), FK): Kedvelő felhasználó azonosítója
    - postId (varchar(191), FK): Kedvelt poszt azonosítója

revokedtoken

- Leírás: Visszavont hozzáférési tokenek.
- Mezők:
    - id (varchar(191), PK): Egyedi azonosító
    - tokenHash (varchar(191)): Token hash értéke
    - expiresAt (datetime(3)): Token lejárati ideje
    - revokedAt (datetime(3)): Token visszavonásának időpontja


tokenHash
expiresAt
revokedAt
id
revokedToken
authTokenVersion
description
isPrivate
isBlocked
app_id
refresh_token
expires_at
access_token
connected_at
connected_apps
platform
role
updated_at
created_at
password_hash
username
email
user_id
user
user_apps
spotifyTimeRange
tokenHashed
id
passwordResetToken
expiersAt
createdAt
usedAt
postLike
id
user_posts
post_id
like_amount
text
posts
hastags
posted_at
topic
title
user_post_comment
triggered_by
notification_id
get_notification
message
notifications
type
read
created_at
moderationLog
id
action
createdAt
detailes
comments
previous_comment
text
commented_at
commentLike
comment_id
like_amount
id

Algoritmusok

formatRelative (relatív idő formázása)

Ez az algoritmus egy adott dátumot hasonlít össze a jelenlegi idővel, és relatív időformátumban adja vissza az eltelt időt (például percekben, órákban vagy napokban). Ha az eltelt idő kisebb, mint 60 perc, perceket ad vissza, ha kevesebb, mint 24 óra, akkor órákat, ha kevesebb, mint 30 nap, akkor napokat, egyébként a pontos dátumot adja vissza helyi formátumban. Az Intl.RelativeTimeFormat API-t használja a megfelelő nyelvi formázáshoz.

```
const formatRelative = (date: Date) => {
  const now = new Date();
  const posted = new Date(date);
  const diff = (posted.getTime() - now.getTime()) / 1000;

  const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

  const minutes = Math.round(diff / 60);
  const hours = Math.round(diff / 3600);
  const days = Math.round(diff / 86400);

  if (Math.abs(minutes) < 60) return rtf.format(minutes, "minute");
  if (Math.abs(hours) < 24) return rtf.format(hours, "hour");
  if (Math.abs(days) < 30) return rtf.format(days, "day");
  return new Date(posted).toLocaleDateString("sv-SE");
};
```

getNormalizedHashtags (hashtag normalizálása)

Ez az algoritmus egy szöveges bemenetből kinyeri a hashtageket. Először szétbontja a szöveget egy reguláris kifejezés alapján, majd kiszűri az üres elemeket. Ezután minden elemet úgy alakít át, hogy ha nem kezdődik **#** jellel, akkor hozzáadja azt, majd az összes hashtaget kisbetűssé alakítja. Így egységes, normalizált hashtag lista jön létre.

```
export const getNormalizedHashtags = (value: string): string[] =>
  value
    .split(HASHTAG_SPLIT_REGEX)
    .filter(Boolean)
    .map((text) => (text.startsWith("#") ? text : `#${text}`))
    .map((text) => text.toLowerCase());
```

parseCookieHeader (cookie fejléc feldolgozása)

Ez az algoritmus egy cookie fejléc stringet dolgoz fel, és egy kulcs-érték párokat tartalmazó objektumot ad vissza. Először ellenőrzi, hogy van-e cookie fejléc, majd a **;** karakter mentén szétbontja a stringet. Minden szegmenst **=** jel mentén kettéválaszt, és a kulcsot, valamint az értéket eltávolítja a felesleges szóközöktől, majd az értéket URI-dekódolja. Az így kapott kulcs-érték párokat egy objektumba gyűjti, amit visszaad.

```
export const parseCookieHeader = (cookieHeader?: string) => {
  const parsed: Record<string, string> = {};

  if (!cookieHeader) {
    return parsed;
  }

  for (const segment of cookieHeader.split(";")) {
    const separatorIndex = segment.indexOf("=");
    if (separatorIndex === -1) {
      continue;
    }

    const rawKey = segment.slice(0, separatorIndex).trim();
    const rawValue = segment.slice(separatorIndex + 1).trim();

    if (!rawKey) {
      continue;
    }

    parsed[rawKey] = decodeURIComponent(rawValue);
  }

  return parsed;
};
```

getBodyParserFriendlyError (body-parser hibák leképezése és formázása)

Ez az algoritmus egy alkalmazásszintű hibaobjektumot (AppError) dolgoz fel, és egy HTTP státuszkódot, valamint egy felhasználóbarát üzenetet tartalmazó objektumot ad vissza. Először megvizsgálja a kapott hiba típusát (err.type) egy switch utasítás segítségével. Ha a típus "entity.parse.failed", "entity.too.large" vagy "encoding.unsupported", akkor a hiba jellegének megfelelő státuszkódot (400, 413, illetve 415) és egy beszédes, angol nyelvű hibaüzenetet párosít hozzá, majd ezt az új objektumot adja vissza. Bármilyen más, fel nem ismert hibatípus esetén alapértelmezettként null értékkel tér vissza.

```
const getBodyParserFriendlyError = (err: AppError) => {
  switch (err.type) {
    case "entity.parse.failed":
      return {
        statusCode: 400,
        message: "The request body is not valid JSON.",
      };
    case "entity.too.large":
      return {
        statusCode: 413,
        message: "The request body is too large.",
      };
    case "encoding.unsupported":
      return {
        statusCode: 415,
        message: "The request body uses an unsupported encoding.",
      };
    default:
      return null;
  }
};
```

Tesztelés

Login teszt (POST /login)

Ez a teszt azt ellenőrzi, hogy a bejelentkezés sikeresen működik-e. A jelszót bcrypt-tel hash-eli, majd egy mockolt adatbázisból lekéri a felhasználót. Ezután elküldi a bejelentkezési kérést a szervernek, és ellenőrzi, hogy a válasz státusza 200, az üzenet "Login successful", valamint, hogy a válasz fejlécében beállításra kerül egy "beatwave_token" nevű cookie.

```
it("GET /user-profile returns profile data for authenticated user", async () => {
  allowAuthenticatedUser("user-1");
  prismaMock.user.findUnique.mockResolvedValueOnce({
    id: "user-1",
    username: "alice",
    email: "alice@example.com",
    description: "About Alice",
    isPrivate: false,
    role: "USER",
    spotifyTimeRange: "MEDIUM",
    connectedApps: [],
  });

  const res = await request(app)
    .get("/user-profile")
    .set("Cookie", makeAuthCookie("user-1"));

  expect(res.status).toBe(200);
  expect(res.body).toEqual(
    expect.objectContaining({
      id: "user-1",
      username: "alice",
      spotifyConnected: false,
      soundCloudConnected: false,
    }),
  );
});
```

Regisztrációs teszt (POST /register)

Ez a teszt egy új felhasználó létrehozását vizsgálja. Mockolt adatbázisban előkészít egy új felhasználói adatot, majd elküldi a regisztrációs kérést. Ellenőrzi, hogy a válasz státusza 201, és hogy a visszakapott felhasználói adatok (felhasználónév, e-mail) helyesek, valamint, hogy a jelszó hash nem kerül visszaadásra (biztonsági okokból).

```
it("POST /register creates a user", async () => {
  prismaMock.user.findFirst.mockResolvedValueOnce(null);
  prismaMock.user.create.mockResolvedValueOnce({
    id: "user-1",
    username: "alice",
    email: "alice@example.com",
    passwordHash: "hashed-password",
    role: "USER",
    authTokenVersion: 0,
    createdAt: new Date(),
    updatedAt: new Date(),
  });

  const res = await request(app).post("/register").send({
    username: "alice",
    email: "alice@example.com",
    password: "Password1!",
  });

  expect(res.status).toBe(201);
  expect(res.body.username).toBe("alice");
  expect(res.body.email).toBe("alice@example.com");
  expect(res.body.passwordHash).toBeUndefined();
});
```

Felhasználói profil lekérdezés (GET /user-profile)

Ez a teszt azt ellenőrzi, hogy egy hitelesített felhasználó profiladatai helyesen visszaadódnak-e. Mockolt adatbázisból lekéri a felhasználó adatait, majd elküldi a profil lekérdezési kérést egy hitelesítő cookie-val. Ellenőrzi, hogy a válasz státusza 200, és hogy a visszakapott profiladatok megfelelnek a várt értékeknek (pl. felhasználó azonosító, név, kapcsolt alkalmazások státusza).

```
it("POST /login authenticates and sets auth cookie", async () => {
  const plainPassword = "Password1!";
  const passwordHash = await bcrypt.hash(
    `${plainPassword}${process.env.PASSWORD_PEPPER}`,
    12,
  );

  prismaMock.user.findFirst.mockResolvedValueOnce({
    id: "user-1",
    username: "alice",
    email: "alice@example.com",
    role: "USER",
    authTokenVersion: 0,
    passwordHash,
  });

  const res = await request(app).post("/login").send({
    login: "alice@example.com",
    password: plainPassword,
  });

  const setCookie = getSetCookieHeader(
    res.headers as Record<string, unknown>,
  );

  expect(res.status).toBe(200);
  expect(res.body.message).toBe("Login successful");
  expect(
    setCookie?.some((cookie) => cookie.startsWith("beatwave_token=")),
  ).toBe(true);
});
```

Új hozzászólás létrehozása (POST /posts)

Ez a teszt azt ellenőrzi, hogy egy hitelesített felhasználó képes-e új hozzászólást létrehozni. Mockolt adatbázisban előkészít egy új posztot, majd elküldi a létrehozási kérést a szervernek. Ellenőrzi, hogy a válasz státusza 201 (sikeres létrehozás), és hogy a visszakapott poszt azonosítója megfelel a várt értéknek.

```
it("POST /posts creates a discussion post", async () => {
  allowUserToContribute("user-1");

  prismaMock.post.create.mockResolvedValueOnce({
    id: "post-1",
    title: "Test title",
    topic: "General",
    text: "Test post body",
    hashtags: "#test",
    userId: "user-1",
    user: {
      id: "user-1",
      username: "alice",
    },
  });

  const res = await request(app)
    .post("/posts")
    .set("Cookie", makeAuthCookie("user-1"))
    .send({
      title: "Test title",
      topic: "General",
      text: "Test post body",
      hashtags: "#test",
    });

  expect(res.status).toBe(201);
  expect(res.body.id).toBe("post-1");
});
```

Jelszó visszaállítási kérés ismeretlen e-mailre (POST /auth/password-reset/request)

Ez a teszt azt vizsgálja, hogy ha egy ismeretlen e-mail címmel kérnek jelszó visszaállítást, akkor a szerver egy általános választ ad vissza, amely nem árulja el, hogy az e-mail létezik-e az adatbázisban. Ellenőrzi, hogy a válasz státusza 200, az üzenet tartalmazza, hogy "If an account exists", és hogy nem történik e-mail küldés (mivel az e-mail nem ismert).

```
it("POST /auth/password-reset/request returns generic response for unknown email", async () => {
  prismaMock.user.findUnique.mockResolvedValueOnce(null);

  const res = await request(app).post("/auth/password-reset/request").send({
    email: "unknown@example.com",
  });

  expect(res.status).toBe(200);
  expect(res.body.message).toContain("If an account exists");
  expect(sendTemplatedEmailMock).not.toHaveBeenCalled();
});
```

Hirdetések lekérdezése (GET /announcements)

Ez a teszt azt ellenőrzi, hogy egy hitelesített felhasználó lekérheti-e a hirdetményeket. Mockolt adatbázisból előkészít egy hirdetményt, majd elküldi a lekérdezési kérést. Ellenőrzi, hogy a válasz státusza 200, a visszakapott lista legalább egy elemet tartalmaz, és hogy az adatbázis-lekérdezés a megfelelő szűrőfeltételekkel történt (csak hirdetmény típusú posztokat kér le).

```
it("GET /announcements returns announcement posts", async () => {
  allowAuthenticatedUser("user-1");
  prismaMock.post.findMany.mockResolvedValueOnce([
    {
      id: "announcement-1",
      title: "Maintenance",
      topic: "Announcement",
      text: "Scheduled maintenance tonight.",
      hashtags: null,
      postedAt: new Date(),
      userId: "admin-1",
      user: {
        id: "admin-1",
        username: "admin",
      },
      likes: [],
    },
  ]);

  const res = await request(app)
    .get("/announcements")
    .set("Cookie", makeAuthCookie("user-1"));

  expect(res.status).toBe(200);
  expect(res.body).toHaveLength(1);
  expect(prismaMock.post.findMany).toHaveBeenCalledWith(
    expect.objectContaining({
      where: {
        OR: [{ topic: "Announcement" }, { topic: "announcement" }],
      },
    }),
  );
});
```

Hibakezelés

getUniqueConstraintMessage (egyedi megszorítási hibaüzenet generálása)

Ez az algoritmus egy mezőneveket tartalmazó string tömböt dolgoz fel, és egy felhasználóbarát hibaüzenetet ad vissza (például akkor, ha egy már létező e-mail címmel próbálnak regisztrálni). Először megvizsgálja a tömb hosszát: ha üres, egy általános üzenetet ad vissza. Ha pontosan egy elemet tartalmaz, akkor azt az egy mezőt említi meg az üzenetben. Több elem esetén az utolsó előtti elemig vesszővel, az utolsót pedig "and" kötőszóval fűzi össze, majd ezt az angol nyelvű felsorolást illeszti be a végső, ütközésre figyelmeztető hibaüzenetbe.

```
const getUniqueConstraintMessage = (fields: string[]) => {
  if (fields.length === 0) {
    return "A record with the same details already exists.";
  }

  if (fields.length === 1) {
    return `A record with the same ${fields[0]} already exists.`;
  }

  const listedFields = `${fields.slice(0, -1).join(", ")} and ${fields.at(-1)}`;
  return `A record with the same ${listedFields} already exists.`;
};
```

sanitizeClientMessage (kliensnek szánt hibaüzenet tisztítása és szűrése)

Ez az algoritmus egy HTTP státuszkódot és egy opcionális nyers hibaüzenetet dolgoz fel, majd egy biztonságos, végfelhasználónak is megjeleníthető stringet ad vissza. Először ellenőrzi, hogy szerverhibáról (500 vagy nagyobb) van-e szó; ha igen, a belső hibák elrejtése érdekében egy általános üzenettel tér vissza. Ezután a nyers üzenetet megtisztítja a felesleges szóközöktől. Ha a kapott üzenet üres, túl hosszú (több mint 180 karakter), vagy egy reguláris kifejezés alapján érzékeny technikai/adatbázis információkat (pl. prisma, sql, stack, query) tartalmaz, akkor szintén egy biztonságos, általános hibaüzenetet ad vissza, megakadályozva az adatszivárgást. Ha minden ellenőrzésen átjut, a megtisztított üzenettel tér vissza.

```
const sanitizeClientMessage = (statusCode: number, rawMessage?: string) => {
  if (statusCode >= 500) {
    return "Something went wrong. Please try again later.";
  }

  const message = (rawMessage || "").replace(/\s+/g, " ").trim();
  if (!message) {
    return "Request failed.";
  }

  if (message.length > 180) {
    return "We could not process your request. Please check your input and try again.";
  }

  if (/(prisma|database|sql|query|constraint|p\d{4}|stack)/i.test(message)) {
    return "We could not process your request. Please check your input and try again.";
  }

  return message;
};
```

errorHandler (központi Express hibakezelő middleware)

Ez az algoritmus egy Express alkalmazás központi hibakezelőjeként funkcionál, amely a beérkező hibaobjektumot dolgozza fel, és a megfelelő HTTP választ generálja le. Először megvizsgálja az aktuális futtatási környezetet (fejlesztői vagy éles), előkészíti a megfelelő státuszkódokat (köztük egy kliens-biztos verziót is), majd naplózza a hibát a szerver konzoljára. A válasz tartalma a környezettől függ: ha fejlesztői környezetben fut, akkor a JSON válaszban részletes információkat (üzenet, hibanév, kód, stack trace) küld vissza a könnyebb hibakeresés érdekében. Éles környezetben viszont kizárólag a már biztonságossá tett (clientError) hibaüzenetet küldi el, megvédve a rendszert a belső architektúra felfedésétől.

```
export const errorHandler = (
  err: AppError,
  _req: Request,
  res: Response,
  _next: NextFunction,
) => {
  const isDevelopment =
    (process.env.NODE_ENV || "development") === "development";
  const statusCode = getStatusCode(err.status);
  const clientError = toClientError(err);
  const responseStatusCode = isDevelopment ? statusCode : clientError.statusCode;

  console.error(err);

  if (isDevelopment) {
    return res.status(responseStatusCode).json({
      error: err.message || "Internal Server Error",
      name: err.name,
      code: err.code,
      stack: err.stack,
    });
  }

  return res.status(responseStatusCode).json({
    error: clientError.message,
  });
};
```

Fejlesztési lehetőségek

Képek föltöltése

Jelenleg a felhasználók még nem tudnak képek föltölteni a Discussion oldalon. Célunk, hogy ezt a funkciót minél hamarabb elérhetővé tegyük, ezzel is gazdagítva a beszélgetési élményt, és lehetőséget adva a vizuális tartalmak megosztására.

Apple Music

Tervezzük, hogy a jövőben az Apple Music felhasználók is csatlakoztathassák fiókjukat a platformhoz. Ezáltal a Beatwave még szélesebb körben tudná kiszolgálni a zenehallgatók igényeit, és több platformról gyűjthetőek lennének zenei adatok.

Alkalmazás készítése

Egy dedikált alkalmazás fejlesztése jelentősen megkönnyítené a Beatwave elérését, mivel a felhasználóknak nem kellene kizárólag webes felületen keresztül használniuk a szolgáltatást.

Tesztelői hozzáférés

Felhasználónév	Jelszó
admin1	Adm!n123

Felhasználói dokumentáció

Projekt áttekintése a felhasználók számára

A Beatwave egy modern, zenealapú közösségi platform, amely összekapcsolja a felhasználók zenei ízlését és közösségi élményeit. A platform lehetővé teszi, hogy a Spotify fiókból származó adatok alapján megtekintsék leggyakrabban hallgatott zenéiket, előadóikat és zenei műfajaikat egy személyre szabott, esztétikus felületen. A rendszer folyamatosan frissül, így mindig a legaktuálisabb toplistákat mutatja. Ezen túlmenően a Beatwave közösségi térként is működik, ahol a felhasználók különböző témájú beszélgetésekben oszthatják meg véleményüket kedvenc előadóikról, albumokról vagy zenei trendekről. A chatfunkció gyors, interaktív és személyes kommunikációt tesz lehetővé.

Szükséges hardver és szoftver

Az oldalt úgy terveztük meg, hogy a lehető legrugalmasabb felhasználói élményt nyújtsa. Ennek köszönhetően a Beatwave az összes eszközön – a klasszikus asztali számítógépektől a legújabb okostelefonokig – natív alkalmazás telepítése nélkül is teljes értékűen fut. A zökkenőmentes használathoz kizárólag egy tetszőleges webböngészőre van szükség.

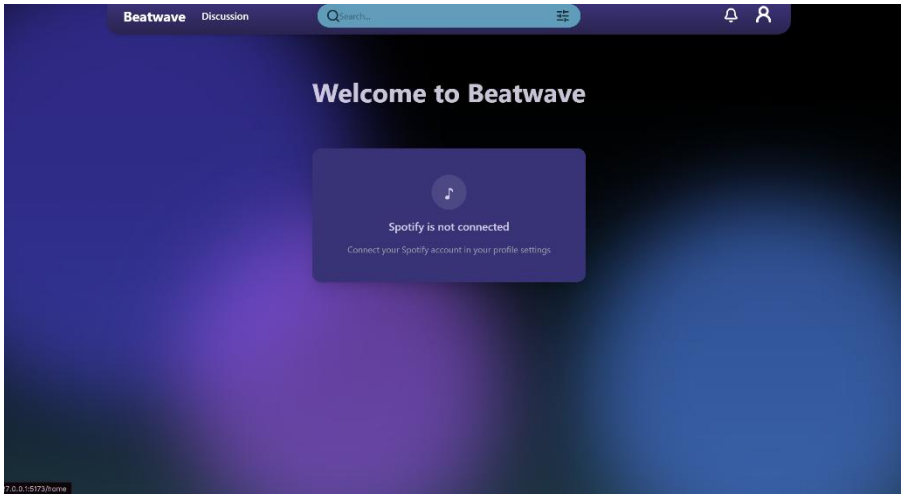
Elérés

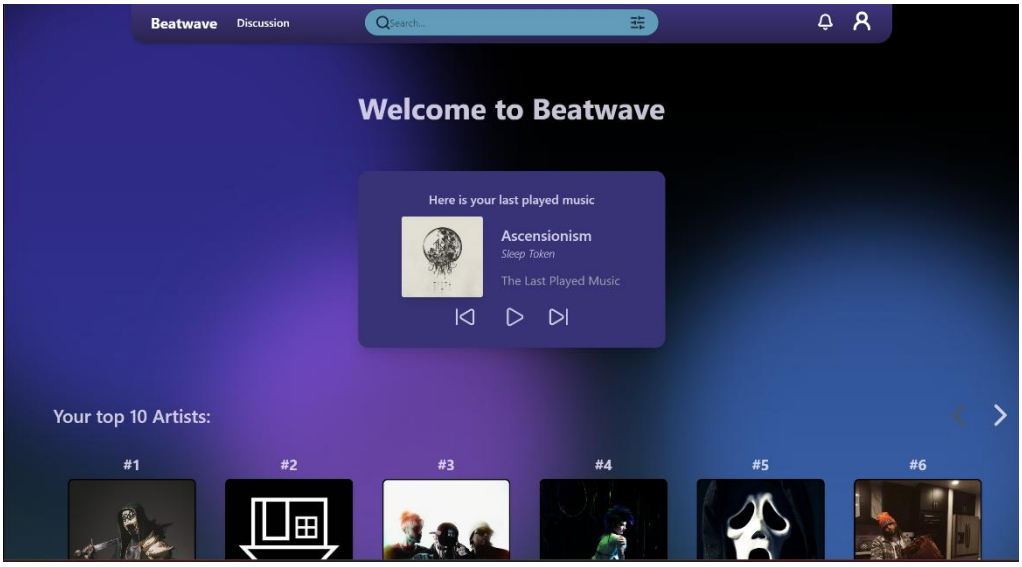
A Beatwave használatához nem kell semmit sem letölteni, mivel egy weboldal. Bármilyen böngészőben a https://beatwave.almavagyok.xyz linken elérhető.

Az oldal használata

A sikeres bejelentkezést követően a felhasználóknak a profilbeállításokban szükséges összekapcsolniuk a fiókjukat a Spotify-jal. Ez a kapcsolat biztosítja, hogy a főoldalon személyre szabott tartalmak jelenjenek meg: a leghallgatottabb előadók és dalok, valamint a legutóbb lejátszott zeneszám, amelynek a lejátszása közvetlenül a weboldalról is vezérelhető. Emellett a rendszer az ízléshez igazodó dalajánlásokat is kínál, amelyeket a felhasználók könnyedén hozzáadhatnak a saját lejátszási listáikhoz. A megjelenített statisztikák időintervalluma szintén a beállításoknál szabható testre; az adatok lekérhetők az elmúlt 4 hétre, 6 hónapra vagy akár egy teljes évre visszamenőleg is.

Az oldalon található beépített kereső alapértelmezés szerint a Spotify adatbázisában keres zenéket, de a szűrők segítségével ez szabadon megváltoztatható. A közösségi élményt a „Discussion” felület biztosítja, ahol a felhasználók saját bejegyzéseket hozhatnak létre, és ezen keresztül kommunikálhatnak egymással. Érdekesség, hogy ezen az oldalon a kereső funkciója is dinamikusan alkalmazkodik a környezethez: itt már nem zenékre, hanem a felhasználói posztok és beszélgetések között lehet vele keresni.





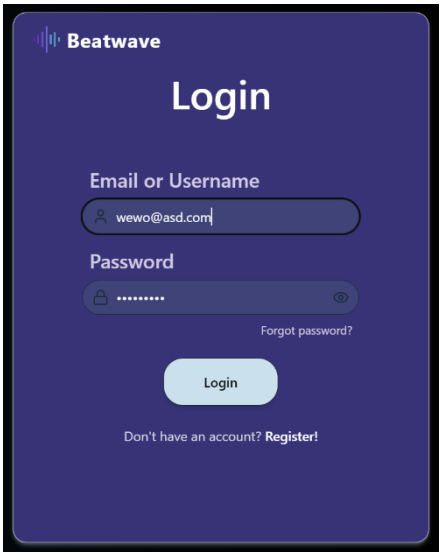
Funkciók elérése

Az oldal funkcióinak eléréséhez a felhasználóknak regisztrálni kell.

Beviteli mezők

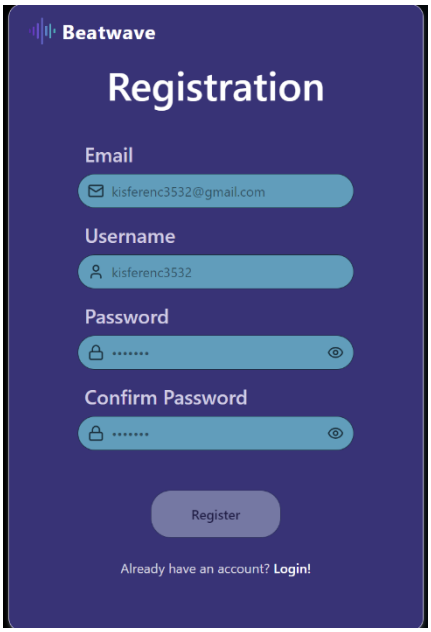
1. Fiók- és bejelentkezési adatok (Authentication Fields)

E-mail cím (Email / Email address)



- **Megjelenés:** Regisztráció, Bejelentkezés, Profil szerkesztése.
- **Elvárt formátum:** Szabványos e-mail cím formátum (**@** jel és domain végződés kötelező, pl. pelda@domain.com).
- **Karakterkészlet:** Kis- és nagybetűk, számok, pont, kötőjel, alulvonás.
- **Ékezetes betűk:** Nem megengedettek.
- **Szóköz:** Nem megengedett.

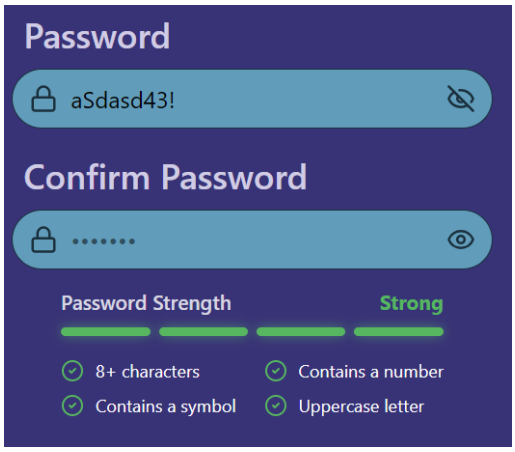
Felhasználónév (Username / New username / Confirm username)



- **Megjelenés:** Regisztráció, Bejelentkezés, Profil szerkesztése.
- **Elvárt formátum:** Egyedi azonosító
- **Karakterkészlet:** Kis- és nagybetűk, számok, speciális karakterként az alulvonás (_) megengedett.
- **Ékezetes betűk:** Az ékezetes betűk meg vannak engedve.
- **Szóköz:** Nem megengedett.
- **Extra szabály:** Profil szerkesztésénél az elgépelések elkerülése végett a megerősítő mezőnek (Confirm username) karakterre pontosan egyeznie kell az új felhasználónévvel.

Jelszó (Password / Confirm Password / Current password)

- **Megjelenés:** Regisztráció, Bejelentkezés, Profil szerkesztése.
- **Biztonsági szabályok (a "Password Strength" indikátor alapján):**
  - **Hosszúság:** Minimum 8 karakter (8+ characters).
  - **Kötelező elemek:**
    - Tartalmaznia kell legalább egy számot (Contains a number).
    - Tartalmaznia kell legalább egy speciális karaktert/szimbólumot (Contains a symbol).
    - Tartalmaznia kell legalább egy nagybetűt (Uppercase letter).
    - (Ebből adódóan kisbetűt is tartalmaznia kell).

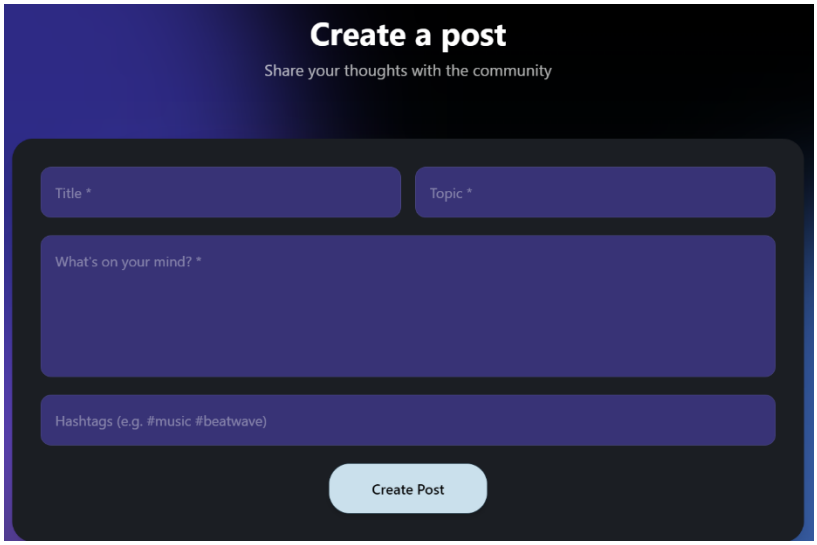


- **Karakterkészlet:** Bármilyen standard billentyűzeten szereplő karakter és ékezetes betű megengedett.
- **Működés:** A beírt szöveg alapértelmezetten maszkolva jelenik meg (pontokkal). A mező végén található "szem" ikonnal a szöveg felfedhető a könnyebb ellenőrzés érdekében. A regisztrációs űrlapon a "Confirm Password" tartalmának teljesen meg kell egyeznie az eredeti jelszóval.

2. Közösségi funkciók (Create a post)

Cím (Title) és Téma (Topic)

- **Kötelezőség:** Mindkettő kitöltése kötelező (a * jelölés mutatja).
- **Karakterkészlet:** Nincs szigorú megkötés. Kis- és nagybetűk, számok, írásjelek egyaránt használhatók.



- **Ékezetes betűk:** Teljes mértékben támogatottak és megengedettek.
- **Szóköz:** Megengedett (többszavas címek és témák).
- **Hossz:** Egysoros, rövid szöveges beviteli mezők.

Bejegyzés szövege (What's on your mind?)

- **Kötelezőség:** Kitöltése kötelező (*).
- **Típus:** Többsoros beviteli mező (textarea).
- **Karakterkészlet:** Bármilyen karakter, írásjel, sortörés (Enter) használható.
- **Ékezetes betűk:** Megengedettek.
- **Hossz:** Szabad szöveges mező, valószínűleg egy magasabb karakterkorláttal (pl. 500-1000 karakter, bár pontos limit a képen nem látszik).

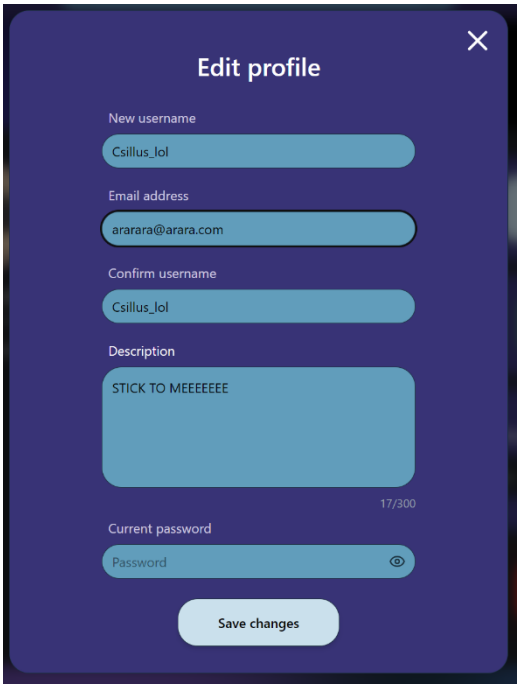
Címkék (Hashtags)

- **Kötelezőség:** Opcionális mező.
- **Elvárt formátum:** A példa (#music #beatwave) alapján kettőskereszttel (#) kezdődő, szóközzel elválasztott szavak.
- **Szóköz:** Csak a hashtagek elválasztására szolgál, magában a címkében nem értelmezhető.

3. Profil személyre szabása (Edit profile)

Leírás / Bemutatkozás (Description)

- **Típus:** Többsoros beviteli mező (textarea).
- **Hosszúság:** Szigorúan limitált. A képernyőképen látható számláló (17/300) alapján **maximum 300 karakter** adható meg.
- **Karakterkészlet:** Szabad szöveg; kis- és nagybetűk, számok, írásjelek, sortörések egyaránt engedélyezettek.
- **Ékezetes betűk:** Megengedettek.



Hibajelzések

Megjelenés és vizuális jellemzők

Kialakítás: A hibaüzenetek piros hátterű, lekerekített formájú (úgynevezett "pill" vagy pirula alakú) felugró értesítésekként (toast/snackbar) jelennek meg.

Ikonográfia: Mindegyik üzenet mellett egy körbe zárt fehér felkiáltójel (!) található, amely az egyezményes vizuális jele a hibáknak és hiányosságoknak, így azonnal megragadja a felhasználó figyelmét.

Elhelyezkedés: Ezek az értesítések általában a képernyő egy jól látható pontján (pl. a tetején vagy az alján) bukkannak fel rövid időre, megakasztva a beküldési folyamatot.

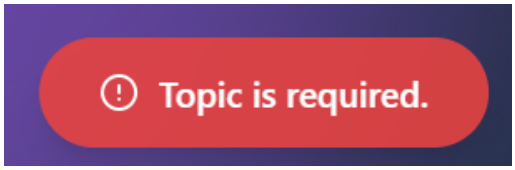
A hibajelzések típusai és kiváltó okaik

Title is required (A cím megadása kötelező)

- Mikor jelenik meg? Ha a felhasználó a "**Title ***" (Cím) mezőt üresen hagyja a poszt létrehozásakor.
- Célja: Biztosítja, hogy a közösségi feedben minden bejegyzésnek legyen azonosítható, kattintható fejléce.

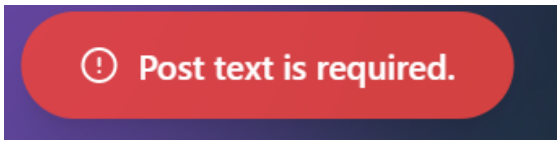
Topic is required (A téma megadása kötelező)

- Mikor jelenik meg? Ha a "Topic *" (Téma) mező kitöltése elmarad.
- Célja: Kikényszeríti a posztok kategorizálását, ami elengedhetetlen a későbbi kereshetőség és a tematikus szűrés szempontjából.



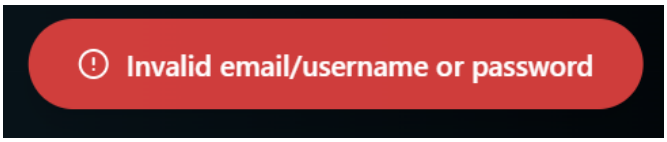
Post text is required (A bejegyzés szövege kötelező)

- Mikor jelenik meg? Ha a "What's on your mind? *" többsoros szövegdoboz üres marad.
- Célja: Megakadályozza az üres, tartalom nélküli posztok (spam) létrejöttét.



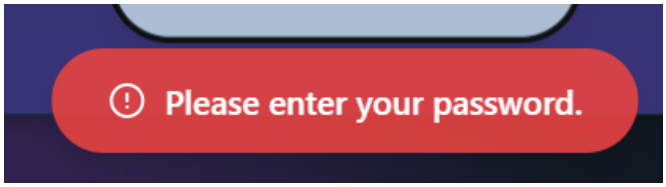
Invalid email/username or password (Érvénytelen e-mail/felhasználónév vagy jelszó)

- Mikor jelenik meg? Ha a megadott hitelesítési adatok nem párosíthatók az adatbázisban tároltakkal.
- Célja: Tájékoztatja a felhasználót a sikertelen bejelentkezési kísérletről.
- Biztonsági szempont: Az üzenet szándékosan nem részletezi, melyik adat a hibás, ezzel megelőzve az *account enumeration* (fiók-kitalálás) típusú támadásokat.



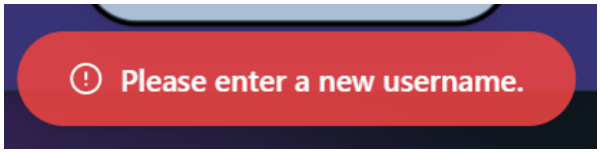
Please enter your password (Jelszó hiánya)

- Mikor jelenik meg? Ha a felhasználó úgy próbál profilmódosítást menteni, hogy a biztonsági ellenőrzéshez szükséges jelszómezőt üresen hagyja.
- Célja: Hitelesíti, hogy a módosítást a fiók tulajdonosa végzi.
- Biztonsági szempont: Megakadályozza, hogy illetéktelenek módosíthassák a profiladatait egy nyitva hagyott eszközön.



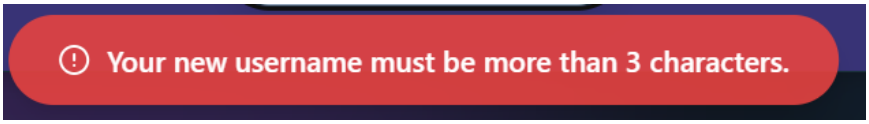
Please enter a new username (Üres felhasználónév)

- Mikor jelenik meg? Ha a felhasználó a mentés gombra kattint, de a felhasználónév mezőt üresen hagyja.
- Célja: Emlékezteti a felhasználót a kötelező mező kitöltésére.
- Felhasználói élmény: Segít elkerülni az érvénytelen adatküldést és közvetlen visszajelzést ad.



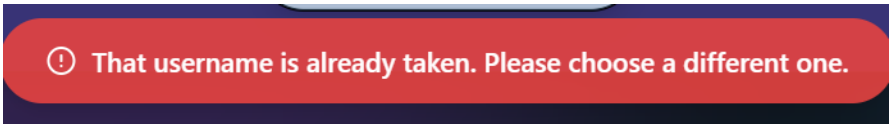
Your new username must be more than 3 characters (Túl rövid felhasználónév)

- Mikor jelenik meg? Ha a beírt név hossza kevesebb mint 3 karakter.
- Célja: Biztosítja a felhasználónevek minimális olvashatóságát és konzisztenciáját.
- Technikai szempont: Segíti az adatbázis-szintű indexelést és megelőzi a technikai rendellenességeket.



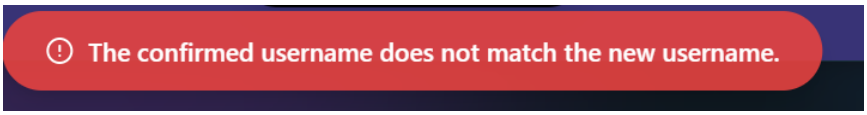
That username is already taken (Foglalt felhasználónév)

- Mikor jelenik meg? Ha a választott név már létezik a rendszerben egy másik fiókhoz rendelve.
- Célja: Biztosítja a felhasználónevek egyediségét.
- Biztonsági szempont: Megakadályozza a fiókok megszemélyesítését (*impersonation*), védi a felhasználói identitást.



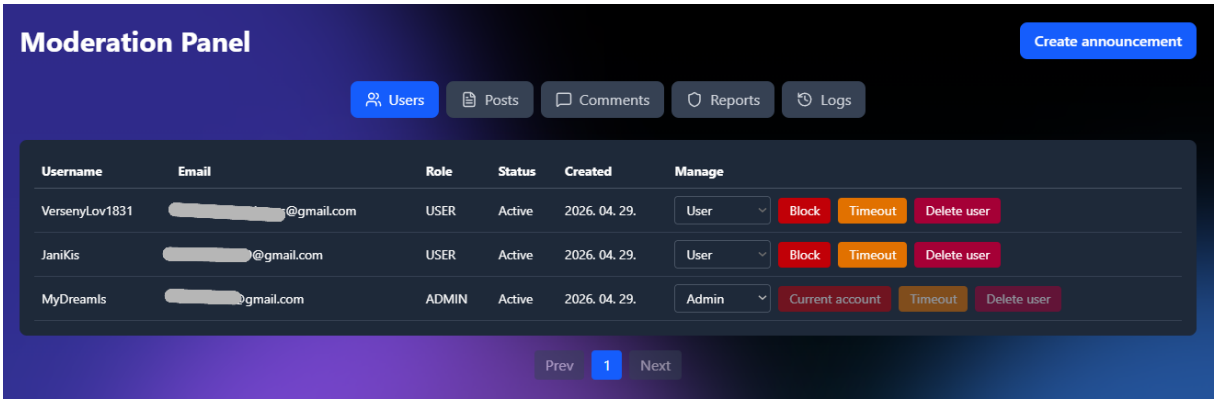
The confirmed username does not match the new username (Felhasználónév-egyeztetési hiba)

- Mikor jelenik meg? Ha az "új felhasználónév" és az "új felhasználónév megerősítése" mezőkbe beírt értékek eltérnek.
- Célja: Megelőzi az elgépelésből adódó hibákat.
- Felhasználói élmény: Hibatűrő megoldás, amely megakadályozza, hogy a felhasználó egy elírt névvel zárja ki magát a saját fiókjából.



Admin lehetőségei

A rendszerben az adminisztrátorok kiemelt jogosultságokkal rendelkeznek. Az adminisztrációs panelen keresztül teljeskörűen kezelhetik a felhasználói fiókokat: módosíthatják a tagok szerepköreit – például moderátorrá léptethetik elő őket –, illetve szabályszegés esetén lehetőségük van a felhasználók kitiltására is.

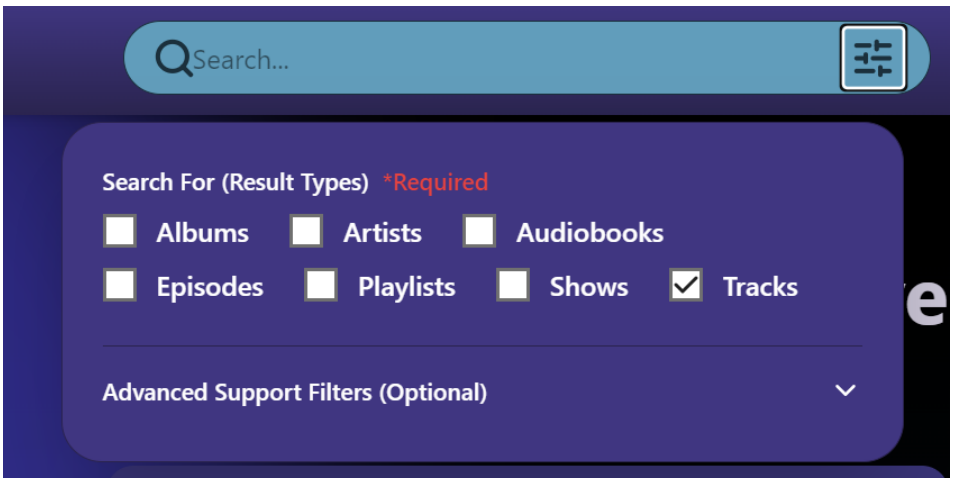


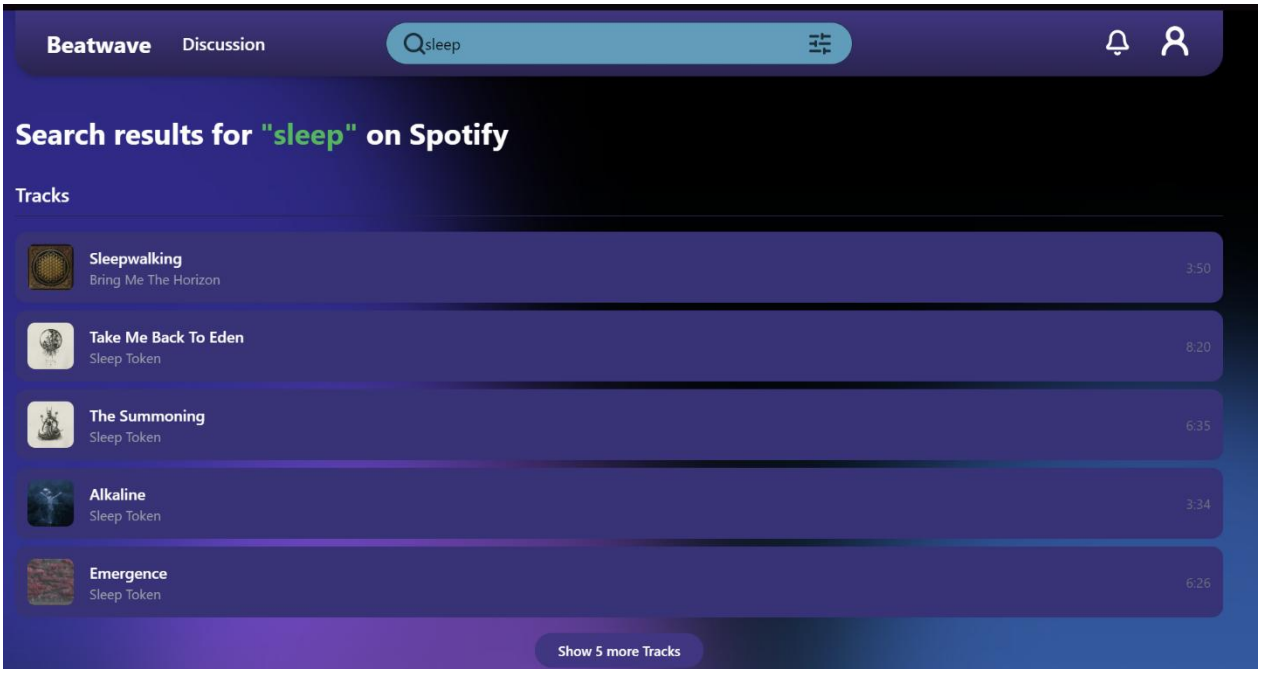
Kereső

Zenei kereső (Spotify integráció)

Ez a kereső a külső zenei adatbázisban böngészik.

- **Funkció:** Zeneszámok, előadók, albumok, podcastok stb. megtalálása.
- **Szűrők:** Lehetővé teszi a találatok típus szerinti leszűkítését (pl. csak a "Tracks" vagy csak az "Albums" legyen látható), ami elengedhetetlen a pontos találatokhoz, hiszen egy keresőszó több kategóriában is releváns lehet.
- **Felhasználói élmény:** A szűrési lehetőségek (Advanced Support Filters) segítségével a felhasználók gyorsan megtalálhatják, amit keresnek, elkerülve a zajos találati listákat.

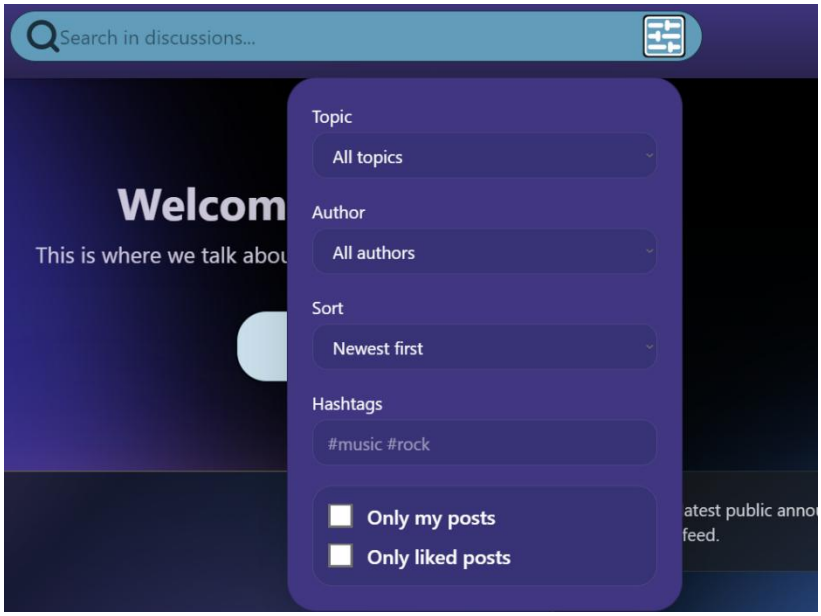


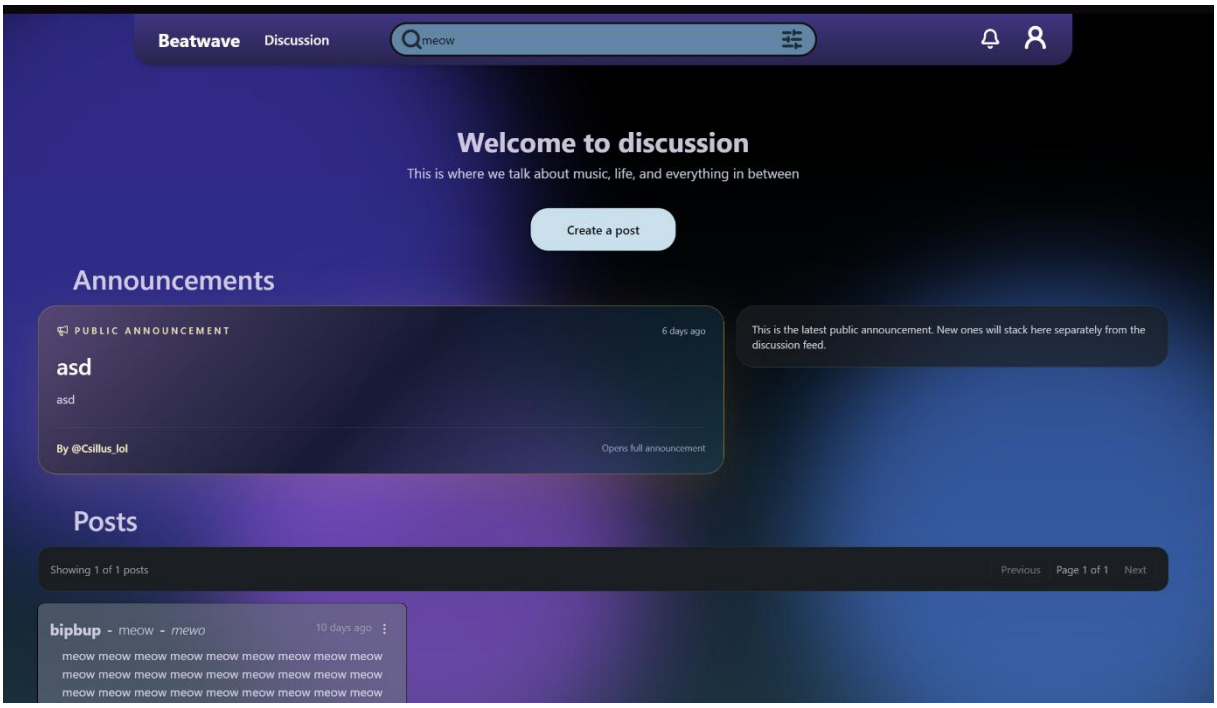


Fórum kereső

Ez a kereső a ti saját közösségi adatbázisotokban, a posztok és beszélgetések között keres.

- **Funkció:** A felhasználók által generált tartalmakban való böngészés.
- **Szűrők:** Mivel itt a közösségi interakció a cél, a szűrők is ehhez igazodnak: **Téma**, **Szerző**, **Rendezési sorrend** (pl. legújabb elöl), **Hashtagek**, valamint a **Saját/Kedvelt posztok** közötti válogatás.
- **Felhasználói élmény:** Ez a kereső segít közösséget építeni, hiszen lehetővé teszi a konkrét beszélgetésekre való rákeresést, vagy a saját aktivitás gyors visszakeresését.




Beatwave
Discussion
meow
Welcome to discussion
This is where we talk about music, life, and everything in between
Create a post
Announcements
PUBLIC ANNOUNCEMENT
6 days ago
asd
asd
By @Csillus_lol
Opens full announcement
This is the latest public announcement. New ones will stack here separately from the discussion feed.
Posts
Showing 1 of 1 posts
Previous
Page 1 of 1
Next
bipbup - meow - mewo
10 days ago
meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow
meow meow meow meow meow meow meow meow meow

Összefoglalás

Értékelés

A Beatwave projektben sikerült megvalósítanunk azt a víziót, ahol a zenei adatvizualizáció harmonikusan találkozik a közösségi élménnyel. Az elkészült platform hűen tükrözi az elmúlt 5 évben megszerzett technikai tudásunkat és annak kreatív alkalmazását. A Spotify API integrációja, a dinamikus statisztikák kezelése, valamint a modern, interaktív kommunikációs modulok fejlesztése nemcsak izgalmas kihívást jelentett, de ugrásszerű szakmai fejlődést is hozott számunkra. Külön sikerként könyveljük el, hogy a nyers adatokból egy személyre szabott, esztétikus és valóban élő digitális teret tudtunk formálni a felhasználók számára. A Beatwave így nem csupán egy kötelező vizsgafeladat, hanem egy olyan komplex referenciamunka, amelyre szakmai és vizuális szempontból is a legbüszkébbek vagyunk, és amely stabil alapot biztosít akár a jövőbeli továbbfejlesztésekhez is.

Irodalomjegyzék

- Vite: https://vite.dev/guide/
- Prisma: https://www.prisma.io/docs
- Express: https://expressjs.com/
- Tailwind: https://tailwindcss.com/docs
- Spotify API: https://developer.spotify.com/documentation/web-api
- EK diagram készítése: https://app.diagrams.net
- Design készítése: https://www.figma.com